# TP 2: Introduction à OpenMP

## Cours de Parallélisme

24 septembre 2025

L'objectif de ce TP est d'utiliser les composants de base d'OpenMP pour créer des sections de code parallèle sans comportement indéterminé.

### Tâche 1

Utilisez OpenMP pour définir une section de code parallèle. Dans cette section, récupérez le numéro du thread en cours et affichez-le. Expliquez le comportement observé.

### Tâche 2

Définissez deux vecteurs : un vecteur contenant des données et un vecteur rempli de 0. Utilisez OpenMP pour paralléliser la copie des données.

#### Bonus

Définissez un entier de taille fixe et un tableau de char non-signé. Utilisez OpenMP pour paralléliser la récupération de chaque bit et leur stockage dans le tableau.
Changez le tableau de char en tableau de booléens, que se passe-t-il ?

### Tâche 3

Complétez le code suivant pour paralléliser la boucle *for* en précisant la visibilité de la variable *seed* :

```cpp
#include <iostream>

void side_effect_function(float& value, size_t index) {
    value = value + index * 0.1f;
}

int main() {
    std::vector<float> data(10, 0.);
    float seed = 7.f;

    // TODO
    for (size_t i = 0; i < data.size(); ++i) {
        side_effect_function(seed, i + 1);
        data[i] = seed;
    }

    for(auto v: data) {
        std::cout << v << " ";
    }
    std::cout << std::endl;
    std::cout << seed << std::endl;
```

```
    return 0;
}
```

Essayez les trois visibilités possible et expliquez les différentes exécutions observées.

Quelle est la visibilité de la variable $i$ dans la boucle parallélisée ?

Quelle est la visibilité par défaut d'une variable :
— Déclarée avant la section parallèle OpenMP, et utilisée dedans ?
— Déclarée et utilisée dans la section parallèle OpenMP ?

## Tâche 4

Définissez un vecteur contenant des valeurs :
— Utilisez OpenMP pour calculer la somme du vecteur.
— Utilisez OpenMP pour obtenir la valeur maximum du vecteur.

### Bonus

Implementez les mêmes opérations en utilisant une section OpenMP critique, ou en utilisant une opération atomique. Mesurez l'impact sur le temps d'éxécution.

## Tâche 5

Dans cette partie vous approximerez pi en utilisant les composants d'OpenMP vu précédemment. L'égalité de Leibniz nous donne que :

$$\frac{\pi}{4} = \arctan(1) = \int_0^1 \frac{1}{1+z^2} dz \tag{1}$$

Vous utiliserez la somme de Riemann permettant d'approximer une intégrale par une somme finie :

$$\int_a^b f(x)dx = \lim_{|\Delta x| \to 0} \sum_{i=1}^{n} f(x_i^*)\Delta x_i \tag{2}$$

Implémentez cette approximation et comparez les temps d'exécution avec et sans OpenMP.
Vous pouvez utiliser le code suivant pour mesurer les performances :

```
#include <chrono>
// ...

const auto start = std::chrono::high_resolution_clock::now();
// Code à mesurer
const auto end = std::chrono::high_resolution_clock::now();
std::cout << "Elapsed time: "
          << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count()
          << " ms" << std::endl;
```